笔杆检测报告单（全文标明引文）

全文标明引文　全文对照　　打印　保存　检测说明

标题：202118010418_PSOS_CourseWork.docx
作者：ZHOU LIPENG
报告编号：BG202505140304575558
提交时间：2025-05-14 03:05:22

去除引用文献复制比：0.2%
去除本人已发表文献复制比：0.2%
单篇最大文字复制比：0.1%

重复字数:48　总字数：23,998　检测范围

202118010418_PSOS_CourseWork.docx_第1部分

原文内容

Principles of Secure Operating Systems - Coursework
Sino-British Collaborative Programme, Oxford Brookes University & Chengdu University of Technology
Semester 2, May 13th,2025
成都理工大学中英合作项目
CHENGDU UNIVERSITY OF TECHNOLOGY SINO-BRITISH COLLABORATIONS
A coursework submitted in partial fulfillment of the requirements for the module - Principles of Secure Operating Systems
Title: Understanding and Modifying An
OS -File Encryption
Name: David (Zhou Lipeng)
Major: BSc (Hons) Computer Science
Student Number:202118010418
Name of Course: Principles of Secure Operating System
Name of Lecturer: Dr. Chiagoziem Chima Ukwuoma
University: Oxford Brookes University
Table of Contents

Chapter 1 Introduction

This file encryption system is designed to provide users with efficient and secure file encryption and decryption functions. The system adopts the AES-256 symmetric encryption algorithm and combines the SHA-256 hash algorithm for file integrity verification to ensure the confidentiality and integrity of the data [1-3]. The system supports multi-threaded encryption and decryption operations to enhance processing efficiency and protects the security of encryption operation permissions through the administrator password mechanism [4-5].

The main functions are shown in the following table:

Function Description

File Encryption/Decryption Support the encryption generation of any file.encrypted files, as well as the decryption and restoration of the original content of encrypted files.

Multithreaded Processing Support concurrent encryption and decryption of multiple files to improve batch processing efficiency.

File Integrity Check

(Hash Verification) Each encrypted file generates and saves a SHA-256 hash value, and its integrity is verified during decryption to prevent tampering.

User Authentication For the first use, an administrator password needs to be set. Subsequent operations require verification to ensure system security.

Key Management Support the secure generation, storage and management of keys to ensure the security of the keys used in encryption and decryption operations.

Friendly Interactive Interface Command-line interaction supports batch file selection and is easy to operate

Table 1□1 Main Functions

This system covers the core requirements of file encryption, including data encryption, integrity verification, permission control and efficient processing, and can provide users with a secure and reliable file protection solution.

Chapter 2 List of Functional and Non-Functional Requirements and security Features of File Encryption

Requirement Description

File Encryption Use the AES-256 encryption algorithm to encrypt files and generate.encrypted files to ensure the confidentiality of data.

File Decryption Be able to decrypt files encrypted with the same algorithm and key/IV, restoring the original content to ensure data availability.

Multithreading Support Utilize multi-threading to concurrently handle the encryption and decryption tasks of multiple files to improve processing performance.

Key and IV Management Support the secure generation, storage and invocation of keys and IVs to prevent leakage and loss.

File I/O Operations Support reading and writing of files, correctly handle file size and location, and be compatible with various file formats.

User Interaction Provide a command-line interface, prompt the user to input necessary information such as operation type, file name, key, etc., and support batch selection of files for easy operation.

Error Handling Provide clear error messages, gracefully handle exceptional situations, and prevent program crashes and data loss.

202118010418_PSOS_CourseWork.docx_第2部分

原文内容

Document Integrity Inspection Generate and save the SHA-256 hash value for each encrypted file, and verify the integrity during decryption to prevent data from being tampered with.

User Authentication Set an administrator password for the first use, and verify the password for subsequent operations to prevent unauthorized access.

Table 2□1 Functional Requirements

Requirement Description

Performance Efficiently utilize system resources, support concurrent processing of large files and multiple files, and minimize processing time as much as possible.

Usability It provides a simple and clear command-line interface, with a clear operation process and explicit error prompts, making it easy for users to get started.

Reliability The program runs stably without crashing or generating damaged output files. All key operations have exception handling.

Scalability The code structure is clear, facilitating the subsequent addition of new functions such as key rotation and log auditing.

Compatibility Supports multiple common file formats and is suitable for different operating system environments.

Maintainability The code comments are sufficient and the structure is reasonable, which is convenient for subsequent maintenance and functional expansion.

Table 2□2 Non-Functional Requirements

Feature Description

Strong Encryption Algorithm It adopts AES-256 symmetric encryption, conforms to industry security standards and is resistant to brute-force cracking.

Random Key and IV Generation Use a secure pseudo-random number generator to generate IVs and keys to ensure security.

Hash Integrity Check The SHA-256 algorithm is used to verify the file integrity to prevent data from being tampered with.

Password Encrypted Storage The administrator password is stored in the form of SHA-256 hash to prevent plaintext leakage.

Key Secure Storage The key and IV are securely stored to prevent leakage and support subsequent encryption and decryption calls.

Permission Control Administrators who only verify through passwords can perform encryption and decryption operations to prevent unauthorized access.

Operation Logs and Exception Handling Key operations and anomalies are all logged and prompted, facilitating tracking and security auditing.

Randomness Guarantee The randomness of IV and keys ensures that encrypted files are difficult to be cracked, enhancing security.

Error Handling A robust error handling mechanism prevents information leakage and system vulnerabilities.

Table 23 Security Features

Chapter 3 Design Of Software That Includes Communications with the OS

3.1Overall Architecture Design

This project consists of two core modules: encryption and decryption, and is jointly supported by various auxiliary components, which form an efficient and secure file encryption system. The functions of each component are summarized as follows:

Module Name Function Description

Encryption Module Encrypt user-selected files using AES-256. Generate random Initialization Vectors (IVs), write IVs to the header of the output file, calculate the SHA-256 hash value of the encryption result, and record it in HashValue.bin to ensure data confidentiality and integrity.

Decryption Module Decrypt encrypted files. Read the IV, decrypt the data, remove padding, and verify the hash value before decryption to prevent tampered files from being misinterpreted as encrypted and enhance data security. If hash verification fails, the user can choose whether to continue decryption.

Key and IV Management Module Manage keys and IVs required for encryption and decryption. Currently, use fixed keys and generate random IVs for each file during encryption. Can be expanded to dynamic key management and key rotation for enhanced security.

Hash Verification Module Generate SHA-256 hash values for encrypted files and save them to HashValue.bin. Verify the hash during decryption to ensure the file has not been tampered with and guarantee data integrity.

Administrator Authentication Module Require administrators to set a password for the first use. Store the password in the form of a SHA-256 hash in adminKey.bin. Verify the password before each operation to prevent unauthorized access.

Multithreading Processing Module Support concurrent encryption and decryption of multiple files. Assign an independent thread to each file for processing, making full use of multi-core CPU resources and improving batch processing efficiency.

File I/O Module Responsible for interactive operations with the operating system, such as file reading, writing, directory traversal, and file deletion, to ensure the efficiency and security of data processing.

Table 3□1 Module Design

Each module collaborates through structures, functions and threads to form a clear hierarchical architecture, which is convenient for maintenance and functional expansion.

3.2Communication with the Operating System

This project communicates with the operating system through multiple standard C libraries and system calls to achieve functions such as file encryption, key management, and multi-threaded processing, as follows:

Function Category Function/Interface Function Description

File System Operations fopen, fread, fwrite, fclose Open, read, write, and close input files, output files, key files, and hash files to ensure the correct access and storage of data.

File System Operations opendir, readdir, closedir Traverse the current directory to filter files eligible for encryption, enabling interaction with the operating system's directory.

File System Operations fseek, ftell Move the file pointer and obtain the file size, supporting block processing and random access of large files.

Random Number Generation RAND_bytes Obtain high-strength random numbers from the operating system for generating encryption keys and Initialization Vectors (IVs), ensuring encryption security.

Encryption Operations AES_set_encrypt_key, AES_cbc_encrypt Implement encryption operations using OpenSSL library functions, relying on the encryption library support of the operating system.

Multithreading Processing mthread_create, mthread_join Create and manage threads to achieve parallel encryption through multithreading, improving processing efficiency.

User Interaction printf, scanf, fgets Interact with users via the command line, receive operation instructions and parameters, and output operation results.

Error Handling and Program Termination fprintf, exit Use fprintf to output error messages to the standard error device; terminate the program when serious errors occur and release system resources through the exit function.

Table 3□2 Functional Details of The Communication Module with the OS

Chapter 4 Implementation Of File Encryption Including Annotated C Code

This project is developed in C language, uses the OpenSSL library to implement AES encryption and decryption, and adopts the mthread library of Minix3.4 to implement multi-threading.

4.1Design and Implementation of Encryption Module

4.1.1Header File Definition

In encrypt.h, the AES block size, parameter structure and function declaration are defined:

These header files provide the necessary functions and data structures for the subsequent implementation of functions such as file encryption, hashing, and threading.

The "Constants" section defines the key parameters and file names used by the encryption program, facilitating unified management and modification.

Define the 'ThreadData' structure to store the data and parameters required by each thread in multi-threaded encryption, covering information such as the source file name, the encrypted output file name, the AES-256 key, and the initialization vector of the CBC mode. Each thread independently completes the corresponding file encryption task according to this structure, achieving centralized parameter management and data isolation among threads.

The remaining part of the header file mainly declares various functional interfaces of the encryption program, covering functions such as error handling, hash calculation, password management, file operation, core encryption, and user interaction.

202118010418_PSOS_CourseWork.docx_第3部分

原文内容

4.1.2Source File Implementation

This module implements the standardized error handling function 'handleErrors', which is used to output detailed error information (including the error file name, line number and error description) when encountering irrecoverable errors during the program execution, and immediately terminate the program execution.

The hash calculation module implements two functions,'sha256' and 'sha256_file', which are respectively used to calculate the SHA-256 hash values of strings and file contents.

This module contains two functions,'set_password' and 'verify_password'.'set_password' is used to set the administrator password and securely write its hash value to the specified file;'verify_password' is used to verify whether the password entered by the user is consistent with the stored hash, ensuring that only authorized users can perform encryption operations.

'save_hash_to_file' is responsible for appending and saving the hash value of the encrypted file in hexadecimal form for subsequent integrity verification.

'save_key_for_file' stores the key corresponding to each encrypted file along with the file name to achieve key management.

'list_files_for_encryption' dynamically traverses the current directory to filter out the ordinary files that can be encrypted for the user to select.

The core encryption module has three components:

'encrypt_file' handles the AES-256 CBC mode encryption, generating keys and IVs, encrypting data blocks, performing padding, and storing keys and hashes.

'encrypt_thread_func' acts as the thread entry point for multi-threaded encryption and calls 'encrypt_file' to execute tasks.

'encrypt_single_file' enables single-file encryption in command-line mode, facilitating scripting or batch processing.

'process_encryption' presents encryptable files, processes user selections, and allocates dedicated threads for encrypting concurrent files to simplify the encryption workflow."print_usage" provides clear program instructions to assist users in their operations.

'main' serves as the entry point of the program, responsible for parsing the command-line parameters, the initial setting and verification of the administrator password, and entering the interactive or command-line encryption process based on the user's choice to ensure the safe and flexible operation of the program.

4.2Design and Implementation of the Decryption Module

4.2.1Header file definition

Similarly, the 'decrypt.h' header file mainly completes the structural definition and functional declaration of the decryption program.

It defines the 'ThreadData' structure required for multi-threaded decryption, centrally manages the input and output file names, keys, and IVs of each thread, and achieves parameter isolation.

The header file also declares various functional interfaces such as error handling, hash calculation, password management, key management, file operation, core decryption, and user interaction.

4.2.2Source file implementation

Similarly, the 'decrypt.c' source file will implement the specific logic of each functional module one by one according to the declaration of the header file.

The error handling module declares the 'handleErrors' function, which is used to standardize the output of error messages and terminate the program.

This module declares' sha256'and' sha256_file ', which are respectively used to calculate the SHA-256 hash values of strings and file contents.

The password management module declares' set_password 'and' verify_password ', which are respectively used to set the administrator password (hash storage) and verify the passwords entered by users, ensuring that only authorized users can perform decryption operations.

The key management module declares' get_key_for_file ', which is used to retrieve the key of the specified encrypted file from the key repository and achieve centralized and secure management of the key.

The file operation module declares' verify_file_hash 'and' list_files_for_decryption '. The former is used to verify the hash value of the decrypted file, and the latter is used to dynamically list the decryptable files in the current directory for the convenience of user selection.

The core decryption module include:

'decrypt_file' implements the specific decryption process of the AES-256 CBC mode.

'decrypt_thread_func' serves as the thread entry for multi-threaded decryption.

'decrypt_single_file' is used for single-file decryption in command-line mode.

The user interaction module declares' process_decryption 'and' print_usage '. The former implements the interactive decryption process, and the latter outputs the program usage instructions to enhance the user experience.

The main function is responsible for parameter parsing, password verification, and process scheduling to ensure the safe and flexible operation of the program.

Chapter 5 Software Verification Test Plan and Results

5.1Software Testing Plan

Test Case ID Test Content Input Data/Operation Expected Result Verification Method

TC-01 Initial Administrator Password Setup Create a new environment and enter the password Generate a password hash file and prompt that the setup is successful Check the file and the output

TC-02 Correct Administrator Password Verification Enter the correct password Pass the verification and allow the operation Observe the program output

TC-03 Incorrect Administrator Password Verification Enter the wrong password Deny access and prompt an error Observe the program output

TC-04 File Encryption Function Select a normal text file for encryption Generate an encrypted file, a key, a hash, and the original file remains unchanged Check the file content and the output

TC-05 File Decryption Function Enter the encrypted file and the key Restore the original file content and output that the decryption is successful Compare the file content

TC-06 Multi-threaded Encryption and Decryption Select multiple files for encryption and decryption simultaneously All files are processed correctly, with no deadlock or crash Conduct concurrent testing

TC-07 Hash Verification Tamper with the encrypted/decrypted file The hash verification fails and prompts an integrity exception Observe the output

TC-08 Key Loss/Error Delete or replace the content of the key library The decryption fails and prompts that the key cannot be found Observe the output

TC-09 Boundary Conditions Encrypt and decrypt an empty file or an extremely large file The program processes normally and gives the correct output Check the file content and the output

TC-10 Illegal Input Use illegal command line parameters or file names The program reports an error and exits without crashing Observe the output

TC-11 Resource Release Check the memory after multiple encryption and decryption operations There is no memory leak Use Valgrind for detection

TC-12 Security Secure storage of passwords, keys, and hashes The file permissions are correct and the content is irreversible Check the file and the permissions

Table 5□1 Test Case Design

5.2Software Test Results

)Before the test, to empty HashValue. Bin, keystore. Bin and adminKey bin directory and prepare the test file 'dog.txt','dog.mp4', and 'hotpot.jpg'.

Figure 5□1 Directory File before the Test

Figure 5□2 Initialization software State

Figure 5□3 Dog.txt

Figure 5□4 Hotpot.jpg

Figure 5□5 Dog.mp4

)Place these test files in a Minix shared folder (/mnt/CW/SE-OS), and display all the files in the current directory by using the "ls" command, as shown in the following figure:

Figure 5□6 Contents of the Minix Shared Folder

)Compile and generate an executable file to test the user verification function (an administrator password needs to be set after initialization).

Figure 5□7 Successfully Compiled the Executable Files Decrypt and Encrypt

Figure 5□8 User Authentication

Figure 5□9 Saves the Password after Setting in adminKey.bin (Hash)

)Use the './encrypt 'command to perform multi-threaded encryption on different types of files (such as' dog.txt','dog.mp4' and 'hot.jpg') simultaneously.

202118010418_PSOS_CourseWork.docx_第4部分

原文内容

Figure 5□10 Multithreaded Encryption

Correspondingly, generate three ciphertext files, as shown in the following figure:

Figure 5□11 Multithreaded Encrypted Ciphertext File

After AES encryption of these three test files, the corresponding ciphertext files of them cannot be opened directly and their contents are unreadable.

Figure 5□12 Original Content of Three Files and Their Corresponding Ciphertext

)Then, use the command './encrypt dog.txt output.txt' to encrypt a single file. As shown in the following figure, the target ciphertext file has been successfully generated.

Figure 5□13 Single-File Encrypted Ciphertext File

Figure 5□14 Comparison of Single-File Encryption

)Now the current directory contains the original file and the ciphertext file, as shown in the following figure:

Figure 5□15 Encrypted Directory Files

)Then use the command "./decrypt output.txt origin.txt "to test the single-file decryption function. The result is shown in the following figure:

Figure 5□16 Decrypted File for Single-File Decryption

After decryption, it can be seen that the decrypted file is exactly the same as the source file.

Figure 5□17 Comparison of the Source File and the Decrypted File

)Use the './decrypt 'command again to test multi-file decryption and decrypt all the decryptable files in the current directory. The result is as follows:

Figure 5□18 Multithreaded Decryption

Figure 5□19 Decrypted Files with Multithreaded Decryption

After decryption, it can be seen that the source file is exactly the same as the decrypted file.

Figure 5□20 Comparison of Source Files and Decrypted Files

Chapter 6 Description Of Integrating the Implemented Component To OS

In the Minix3.4 operating system, the standard directory structure stipulates that 'usr/include' is used to store system-level header files, and 'usr/bin' is used to store executable files for all users. To seamlessly integrate the self-developed encryption/decryption components into the Minix system, this project strictly adheres to this specification: All the header files that implement the encryption/decryption function are migrated to the 'usr/include' directory through the 'mv' command, and then the compiled executable files are migrated to the 'usr/bin' directory. The integration process is shown in Figure 6-1. Through a standardized file deployment strategy, it is ensured that the encryption/decryption tools can be directly called in any working path, achieving the system-level calling capability of the encryption/decryption tools.

Figure 6□1 Integrated Encryption/Decryption Components

Chapter 7 Integration Testing Plan and Results

7.1Integration Test Plan

Test Operation

File Search Test In the Minix3.4 system, execute the "find /-name [filename]" command to retrieve files in the system and verify whether the system can normally recognize and locate the target file.

Function Call Test Try to directly run the encryption/decryption software in any directory of the Minix system using the 'encrypt' and 'decrypt' commands (taking "/usr" as an example).

Table 7□1 Integration Test Plan

7.2Test Results

)Successfully find the files by using the command "find /-name [filename]". The search process is shown in Figure 7-1.

Figure 7□1 Software File Search

)As shown in Figure 7-2, this program can directly call the 'encrypt' and 'decrypt' commands in the '/usr' directory without encryption/software executable files to encrypt/decrypt the files in the current directory, indicating that it can run normally in any directory of the Minix system and indicating that the installation is successful.

Figure 72 Software Verification (Directory '/usr')

Chapter 8 Reports of Possible Limitations, Failures and Encountered Difficulties

This chapter truthfully reflects the main limitations and difficulties encountered during the development process, as shown in Table 8-1:

Issue Category Detailed Description

(a) Cross-platform Compatibility Limitation The project relies on Minix system-specificmthread.hheader for multi-threading, making it incompatible with mainstream operating systems like Linux and Windows. This significantly restricts the software's applicability and fails to meet cross-platform user needs.

(b) Development Environment Adaptation Difficulty Mainstream IDEs like CLion lack support for mthread.h, forcing developers to rely solely on the Minix environment for debugging. The non-intuitive error messages in Minix prolong development cycles and increase technical hurdles.

(c) High Porting & Maintenance Costs Heavy reliance on Minix-specific libraries complicates cross-platform migration. Reconstructing core modules for other systems incurs high costs and maintenance challenges, limiting adaptability to diverse user requirements.

(d) Poor User Experience The command-line interface lacks graphical support and intuitive error handling. Novice users face confusion due to cryptic commands and inadequate guidance, reducing usability.

Table 8□1 Possible Limitations and Difficulties

Chapter 9 Summary and Extended Suggestions

This project designed and implemented the file encryption and decryption software on the Minix 3.4 operating system using the C language. By integrating the AES 256 encryption algorithm in the OpenSSL library and using Minix's proprietary "mthread.h" multi-threaded library, this software can achieve efficient and secure multi-threaded file encryption and decryption operations. Meanwhile, the software is integrated into the Minix operating system, allowing users to access encryption and decryption functions from any location in the system. Rigorous testing has verified the stability of the software, confirming that it meets the expected security and usability requirements.

During the development process, this project identified and resolved multiple challenges such as multi-threaded compatibility, platform dependence, and key management. However, the software still faces limitations, including restricted cross-platform compatibility and the risk of key loss. To enhance its performance, future improvements should focus on adopting widely used third-party multithreading libraries, such as POSIX pthread or the C11 standard thread library, to support mainstream operating systems. Furthermore, implementing a robust key backup and recovery mechanism will reduce the risk of data loss, while introducing a graphical interface with comprehensive error prompts will significantly improve the user experience and lower the entry threshold for ordinary users.

Chapter 10 Explanation and Reflection on Improvement and Revision

This chapter reviews the major improvements in the current project version and analyzes their effectiveness.

Firstly, in order to solve the problems of multi-threading compatibility and platform dependence in the early version, the structure of the multi-threading implementation was optimized. By reasonably encapsulating the thread creation and management logic, the maintainability and scalability of the code have been enhanced, facilitating its future transplantation to other operating systems. Although the 'mthread.h' of Minix is still used, the code structure is clearer and easier to be replaced with the standard thread library later.

Secondly, the key and hash management mechanisms have been greatly improved. In the new version, key files and hash values are stored in a more standardized binary format, and the uniqueness check of key entries has been added, reducing the risk of key conflicts and loss, thereby enhancing data security and system robustness.

Furthermore, in terms of user experience, the modified version has optimized the command-line interaction process. Add detailed error prompts and operation feedback to enable users to quickly understand the execution results and potential problems, strengthen the input verification logic, and improve the system's tolerance for abnormal input.

In conclusion, these improvements not only addressed early issues but also made progress in terms of security, availability, and maintainability, laying a solid foundation for future functional expansion and platform adaptation.

202118010418_PSOS_CourseWork.docx_第5部分

原文内容

References

[1]N. Cherrid, R. Saidi, T. Bentahar, and H. Mayache, "Study of the Sensitivity of an InSAR Interferogram Encrypted by the AES-128 Algorithm," in 2021 18th International Multi-Conference on Systems, Signals & Devices (SSD), Monastir, Tunisia, pp. 457–462, 2021. https://doi.org/10.1109/SSD52085.2021.9429323.

[2]S. Liu, Y. Li, and Z. Jin, "Research on Enhanced AES Algorithm Based on Key Operations," in 2023 IEEE 5th International Conference on Civil Aviation Safety and Information Technology (ICCASIT), Dali, China, pp. 318–322, 2023. https://doi.org/10.1109/ICCASIT58768.2023.10351719.

[3]N. J. R, R. Patil, O. Sawant, S. Madasamy, and S. R, "AES Algorithm with Dynamic Shift Rows and Bit Permuted Mix Column," in 2023 International Conference on Next Generation Electronics (NEleX), Vellore, India, pp. 1–6, 2023. https://doi.org/10.1109/NEleX59773.2023.10421625.

[4]U. S, V. N R, P. N R, and L. S, "Secure Data Transmission using Hybrid Crypto Processor based on AES and HMAC Algorithms," in 2023 2nd International Conference on Advancements in Electrical, Electronics, Communication, Computing and Automation (ICAECA), Coimbatore, India, pp. 1–6, 2023. https://doi.org/10.1109/ICAECA56562.2023.10200277.

[5]D. M, V. B, A. E, A. S, D. S, and S. K. A, "An Efficient Implementation of AES Algorithm for Cryptography Using CADENCE," in 2023 7th International Conference on Electronics, Communication and Aerospace Technology (ICECA), Coimbatore, India, pp. 1478–1484, 2023. https://doi.org/10.1109/ICECA58529.2023.10395793.